

4

Knowledge

4.1 Logical Agents, Knowledge-Based Agents, The Wumpus World, Logic, Propositional Logic - A Very Simple Logic, Propositional Theorem Proving, Effective Propositional Model Checking, Agents Based on Propositional Logic

Q.1 How knowledge can be representation ? Explain with example.

[SPPU : Dec.-16]

Ans. : Knowledge Representation : In toy problems the choice of representation is not important because these problems have consistent vocabulary.

In complex real world problems such as shopping on internet or robot in continuous environment, we require more general and flexible representations. We need to create representations which concentrate on general concepts such as Actions, Time, Physical Objects and Beliefs which can occur in many different problem domains.

Q.2 Represent following facts using propositional logic.

- All pompeian are Romans.
- Caesar was a ruler.
- All Romans were either loyal to Caesar or hated him.
- Everyone is loyal to someone.
- People only try to assassinate rulers they are no loyal.
- Marcus tried to assassinate ceasar.

[SPPU : Oct.-16, Marks 6]

Ans. : i) All pompeians were Romans

$\forall x : \text{Pompeian}(x) \rightarrow \text{Roman}(x)$

In this presentation universal quantifier $\forall$ is used to present universal value of x . i.e., for all x in universe.

Pompeian (x) indicates that x is a pompeian. Next symbol ' $\rightarrow$ ' implies i.e., means and roman (x) indicates that x is Roman.

In other words we say that all x i.e., all pompeians were Roman.

- ii) Caesar was a ruler
ruler (Caesar)

Here, one can ignore the fact that proper names are often not references to unique individuals, since many people share the same name. Sometimes deciding which of several people of the same name is being referred to in a particular statement may require a fair amount of knowledge and reasoning.

- iii) "All romans were either loyal to Caesar or hated him"

$\forall x : \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$

In English, the word "OR" sometimes means the logical inclusive-OR and sometimes means the logical exclusive-OR (XOR).

Here, inclusive interpretation is used.

But, if one considers it as 'exclusive-OR' then presentation would be some what different as follows -

$\forall x : \text{Roman}(x) \rightarrow [\text{loyalto}(x, \text{Caesar}) \vee$
 $\text{hate}(x, \text{Caesar})) \wedge$

$\neg (\text{loyalto}(x, \text{Caesar}) \wedge \text{hate}(x, \text{Caesar}))]$

- iv) "Everyone is loyal to someone"

$\forall x : \exists y : \text{loyalto}(x, y).$

Here, one can use both the quantifiers i.e. $\forall \rightarrow$ universal quantifier and $\exists \rightarrow$ existential quantifier.

So, one can read above presentation as for all x , there exists a y such that x is loyal to y or indirectly we say

"Everyone is loyal to someone".

But here one can understand a different meaning i.e. there is someone to whom everyone is loyal, which would be written as

$\exists y : \forall x : \text{loyalto}(x, y)$

Now read it as, there exists a y , such that for all x , x is loyal to y .

Or

"everyone is loyal to y ".

- v) "People only try to assassinate rulers they are not loyal to"

$\forall x : \forall y : \text{person}(x) \wedge \text{ruler}(y) \wedge$
 $\text{tryassassinate}(x, y) \rightarrow \neg \text{loyal to}(x, y)$

This sentence, too, is ambiguous. Does it mean that the only rulers that people try to assassinate are those to whom they are not loyal, one above presented or does it mean that the only thing people try to do is to assassinate rulers to whom they are not loyal?

vi) "Marcus tried to assassinate Caesar"
 $\text{tryassassinate}(\text{Marcus}, \text{Caesar})$

Q.3 What is propositional logic? Explain with example.

[SPPU : May-18]

Ans. : Propositional Logic :

Characteristic of Propositional Logic :

1. Propositional logic is declarative (pieces of syntax corresponds to facts).
2. Propositional logic allows partial/disjunctive/negated inference (unlike most data structures and databases)
3. Propositional logic is compositional.
 (Meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and meaning of $P_{1,2}$).
4. Meaning in propositional logic is context independent (unlike natural language, where meaning depends on context).

Drawbacks of Propositional Logic :

1. Propositional logic has very limited expressive power (unlike natural language).
 Example : Cannot express "pits causes breezes in adjacent squares".
 For such statement one need to write, one sentence for each square.
2. Propositional logic represents statements about the world without reflecting the structure and without modeling these entities explicitly.
3. Therefore some knowledge is hard or impossible to encode in the propositional logic.
4. Two cases that are hard to represent : -

Case I) Statements about similar objects and their relations :

- a) Statements about similar objects and relations needs to be enumerated.
- b) Because of this knowledge-base grows large.

c) We need to represent many rules to allow inferences. The solution to this problem is introduce variables.

Case II) Statements referring to groups of objects :

Statements referring to groups of objects require exhaustive enumeration of objects.

Solution to this problem is to allow quantification in statements. [which is done in first order logic].

Following are Some of the Legal Sentences (WFF's) in Propositional Logic

1) The negation of a sentence is a sentence.

For example :

$\neg P$ and $\neg \text{false}$ are sentences.

2) The conjunction, 'and', of two sentences is a sentence.

For example :

$P \wedge \neg P$ is a sentences.

3) The disjunction, 'or', of two sentences is a sentence.

For example :

$P \vee \neg P$ is a sentence.

4) The implication of one sentence from another is a sentence.

For example :

$P \rightarrow Q$ is a sentence.

5) The equivalence of two sentences is a sentence.

For example :

$P \vee Q \equiv R$ is a sentence.

Q.4 Explain syntax of defining sentences in propositional logic.

Ans. :

Syntax : It defines allowable sentences in the model.

Propositional calculus symbols :

1. Symbols

i) They are prepositional calculus symbols

$P, Q, R, S \dots$

ii) Predicate symbols : They are used to represent a relation in a domain.

For example :

If we want to represent a sentence like, "Dipu reads book".

We will use the predicate symbol "READS". The simple predicate will be,

READS (Dipu, Book).

- iii) Constant symbols : A constant symbol is used to represent objects or entities in a domain. These objects or entities may be physical objects, people, concepts or anything that is to be named.

For example : In the above formula Dipu and Book are constant symbols.

- iv) Variable symbols : Variables like x or y are also symbols. They allow the users to be indefinite about which entity is to be referred to example READ (x, y).

- v) Function symbols : They denote functions in the domain of discourse.

For example : "Ram's mother is married to Ram's father", the atomic formula would be,

MARRIED [MOTHER (Ram), FATHER (Ram)] where MOTHER, FATHER and MARRIED are function symbols.

2. Truth symbols

True, False.

3. Connectives

$\wedge, \vee, \neg, \rightarrow, \equiv$

Note : Propositional symbols denote propositions, or statements about the world that may be either true or false, such as "the car is red" or "water is wet".

Q.5 What are propositional calculus sentences ?

Ans. :

- Every propositional symbol and truth symbol is sentence.
For example : True, P, Q, R are sentences.
- Atomic sentence : It is indivisible (non-composite) syntactic element. It consists of proposition symbol. Each such symbol stands for a proposition that can be true or false.
- Complex sentence : It is constructed from simpler sentences using logical connectives.

Note : Logical sentences are also called as well-formed formulas or WFF's.

Q.6 Explain inference algorithm in propositional logic.

Ans. : A logical inferencing means to decide whether $KB \models \alpha$ for some sentence α .

General Inferencing algorithm (truth table enumeration algorithm)

- This algorithm is used for deciding entailment in propositional logic.
- It works like backtracking search algorithm.
- It performs a recursive enumeration of a finite space of assignments to variables.
- This algorithm is sound, because it implements directly the definition of entailment.
- This algorithm is complete, because it works for any knowledge base and ' α ', and always terminates (As there are finite models to check).
- Time complexity of algorithm is $O(2^n)$ (assuming n symbol in all, then there are 2^n models).
- Space complexity of algorithm is $O(n)$, because the enumeration is depth first.

Note : Every known inference algorithm for propositional logic has a worst case time complexity which is exponential in the size of the input.

Note : Propositional entailment is CO-NP-complete. (CO-NP stands for Complement - Nonpolynomial time, where as CO-NP-complete means hardest problem in CO-NP)

The functions for truth table enumeration algorithm :

Function TT Entails (KB, α) Returns True or False

Inputs : (KB, the knowledge base),

(a sentence in propositional logic α),

the query, a sentence in propositional logic.

Symbols $\leftarrow$ A list of the proposition symbols in KB
and α

return TT CHECK-ALL (KB, α , symbols, [])

Function TT-CHECK-ALL (KB, α , symbols, model)
returns true or false

```

if EMPTY (symbols) then
  if PL-TRUE (KB, model) then return
    PL-TRUE ( $\alpha$ , model)
  else return true
else do
  P  $\leftarrow$  FIRST (symbols); rest  $\leftarrow$ 
    REST (symbols)
  return TT-CHECK-ALL (KB,  $\alpha$ , rest,
    EXTEND (P, true, model) and
    TT-CHECK-ALL(KB,  $\alpha$ , rest, EXTEND
      (P, false, model)

```

Note that,

- 1) In truth-table enumeration algorithm for deciding propositional entailment, TT stands for truth Table.
- 2) PL-TRUE returns true; if a sentence holds within a model.
- 3) The variable model represents a partial model-an assignment to only some of the variables.
- 4) The function call EXTEND (P, true, model) returns a new partial model in which P has the value true.

Q.7 Represent Wumpus world using propositional logic along inference process.

OR Write the algorithm for Wumpus world agent to find pits, wumpses and safe squares.

OR Explain inference based agent in Wumpus world based on propositional logic.

Ans. : A wumpus world agent :

Wumpus World is a classic artificial intelligence problem, which is used to demonstrate various aspects based on simulation, as well as other AI concepts.

Wumpus was an early computer game in which an agent had to explore a cave made up from a series of interconnected rooms. In one of the rooms

in the cave, there was a Wumpus which would kill the agent if it entered that room. Some rooms contained pits, and the agent would die if it entered any of those rooms too. The agent had one arrow with which it could kill the Wumpus. The goal was to locate the gold that was hidden some where in the cave and return to the start without getting killed.

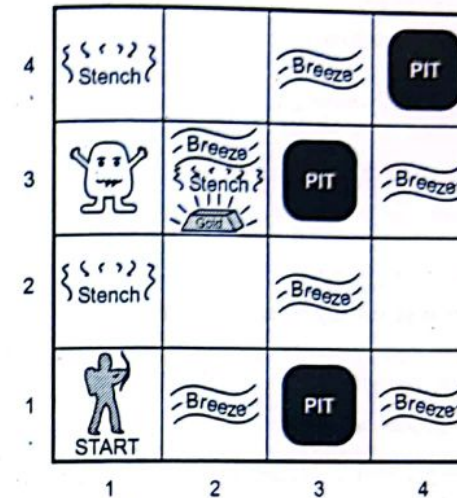


Fig. Q.7.1 A wumpus world agent and its environment
PEAS description :

• **Performance measure :**

Gold : + 1000, Death : - 1000

- 1 per step, - 10 for using the arrow.

• **Environment :**

- Squares adjacent to Wumpus are smelly.
- Squares adjacent to pit are breezy.
- Glitter iff gold is in the same square.
- Shooting kills Wumpus if you are facing it. It screams.
- Shooting uses only arrow.
- Grabbing picks up gold if in same square.
- Releasing drops the gold in same square.
- You bump if you walk into a wall.

• **Actuators :**

Left turn, right turn, forward, grab, release, shoot.

• **Sensors :**

Stench, breeze, glitter, bump, scream.

Wumpus world characterization :

1) Deterministic

Yes - outcomes exactly specified.

2) Static

Yes - Wumpus and Pits do not move.

3) Discrete

Yes

4) Single - agent

Yes - Wumpus is essentially a natural feature.

5) Fully observable

No - only Local perception.

Exploring the wumpus world :

1) The knowledge base initially contains the rules of the environment.

2) Location : [1, 1]

Percept : [$\neg$ Stench, $\neg$ Breeze, $\neg$ Glitter, $\neg$ Bump, $\neg$ Scream]

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
$\boxed{A}$ OK	OK		

$\boxed{A}$ = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. Q.7.2 Exploring wumpus world-I

Action : Move to safe cell [2, 1].

3) Location : [2, 1]

Percept : [$\neg$ Stench, Breeze, $\neg$ Glitter, $\neg$ Bump, $\neg$ Scream].

Infer : Breeze indicates that there is a pit in [2, 2] or [3, 1].

Action : Return to [1, 1] to try next safe cell.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
V OK	$\boxed{A}$ B OK	P?	

$\boxed{A}$ = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. Q.7.3 Exploring wumpus world - II

4) Location : [1, 2]

Percept : [Stench, $\neg$ Breeze, $\neg$ Glitter, $\neg$ Bump, $\neg$ Scream].

Infer : Wumpus in [1, 3] or [2, 2]

YET not in [1, 1]

Thus not in [2, 2] or stench would have been detected in [2, 1]

Thus Wumpus is in [1, 3] [2, 2] is safe because of lack of breeze in [1, 2]

Thus pit in [3, 1]

Action : Move to next safe cell [2, 2].

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
W!			
1,2	2,2	3,2	4,2
$\boxed{A}$ S OK	OK		
1,1	2,1	3,1	4,1
V OK	B V OK	P!	

$\boxed{A}$ = Agent
B = Breeze
G = Glitter gold
OK = Soft square
P = Pit
S = Stench
V = Visited
W = Wumpus

Fig. Q.7.4 Exploring wumpus world - III

Inference based Agent in Wumpus World

It uses inference algorithms and knowledge base like a general knowledge based agent.

Wumpus world

- 1) This agent reasons logically about the location of pits (P), Wumpus (W), and safe squares (S).
- 2) It begins with a knowledge base that states the "physics" of the Wumpus world.
- 3) The agent begins its exploration from a square known to be safe.

$$-R_1 : \neg P_{1,1}$$

$$-R_2 : \neg W_{1,1}$$

- 4) For every square $[x, y]$, if the agent perceives breeze....

$$-R_3 -R_{18} : B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$$

- 5) For every square $[x, y]$, if the agent perceives stench....

$$-R_{19} -R_{34} : S_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$$

- 6) How do we express there is exactly one Wumpus ?

We can represent it in 2 sentences,

- a) There is at least one Wumpus $R_{35} :$

$$W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,4}$$

- b) For any two squares, one must be Wumpus-free.

- i) This should be stated for every pair of squares.

- ii) $[n(n-1)]/2$ number of sentences.

$$-R_{36} : \neg W_{1,1} \vee \neg W_{1,2}$$

$$-R_{37} : \neg W_{1,1} \vee \neg W_{1,3}$$

:

:

$$-R_{155} :$$

- 7) We have used 64 distinct symbols in the knowledge base.

Wumpus-World-Agent program

- 1) ASK-Entailment computation :

TT-Entails would have to enumerate 2^{64} rows.

DPLL/WALKSAT performs better. DPLL works because it uses unit propagation heuristic. WALKSAT suffers from incompleteness.

- 2) For a large Wumpus world, the number of sentences in the knowledge base will be huge.

- 3) We could not capture the property "breeze in a square indicates pit in at least one of directly adjacent square".

Location, orientation in wumpus world

- 1) Can we add propositions like,

$$L_{1,1} \wedge \text{Facing Right} \wedge \text{Forward} \rightarrow L_{2,1}$$

- 2) Knowledge base will entail both $L_{1,1}$ and $L_{2,1}$ by using inference rules.

- 3) What we intended to capture is this

$$L_{1,1}^{(t)} \wedge \text{Facing Right} \wedge \text{Forward} \rightarrow L_{2,1}^{(t+1)}$$

$$\text{Facing Right} \wedge \text{Turn Left}^{(t)} \rightarrow \text{Facing Up}^{(t+1)}$$

- 4) For every times step, one such statement ???

Assuming the problem is solvable in 100 times steps, then 10 thousands of additional sentences are needed.

Q.8 Explain how to have inference using propositional logic. Also explain resolution using propositional logic.

Ans. : 1) Modus ponens

It is represented as,

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

It means that whenever any sentences of the form $\alpha \Rightarrow \beta$

and α are given then the sentence β can be inferred.

For example -

If two statements,

- i) (Signal Pole Ahead $\wedge$ Signal Red) $\Rightarrow$ Stop

- ii) (Signal Pole Ahead $\wedge$ Signal Red) are given then "Stop" can be inferred.

- 2) And-elimination : It says that from conjunction any conjuncts can be inferred.

It is represented as,

$$\frac{\alpha \wedge \beta}{\alpha}$$

For example -

From the sentence,

(Signal Pole Ahead $\wedge$ Signal Red)

The inference,

"Single Red".

can be drawn.

3) Unit resolution [$P \vee Q, \neg Q \vdash P$]:

Unit resolution rule takes a clause - a disjunction of literals and a literal and produce a new clause.

$$\text{Rule is, } \frac{l_1 \vee \dots \vee l_k, m}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k}$$

where each l is literal. l_i and m are complementary literals. [It means that one is the negation of other].

4) Resolution: The unit resolution rule can be generalized to the full resolution rule,

$$\frac{l_1 \vee \dots \vee l_k, m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where l_i and m_j are complementary literals. If we are dealing only with clauses of length two we could write this as

$$\frac{l_1 \vee l_2, \neg l_2 \vee l_3}{l_1 \vee l_3}$$

That is, resolution takes two clauses and produces a new clause containing all the literals of the two original clauses except the two complementary literals.

It is a single inference rule, that yields a complete inference algorithm when coupled with any complete search algorithm.

Conjunctive normal form

- 1) The resolution rule applies only to disjunctions of literals, so it would seem to be relevant only to knowledge bases and queries consisting of such disjunctions.
- 2) Every sentence of propositional logic is logically equivalent to a conjunction of disjunctions of literals.

BRUNN

A Guide for Engineering Students

- 3) A sentence expressed as a conjunction of disjunctions of literals is said to be in Conjunctive Normal Form (CNF).
- 4) It is used for representing sentences.

Steps for converting propositional logic sentences into CNF

- 1) Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$:

For example - Consider following statement,

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

After eliminating $\Leftrightarrow$ in above sentence,

$$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$$

- 2) Eliminate $\rightarrow$, replacing $\alpha \rightarrow \beta$ with $\neg \alpha \vee \beta$:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
- 3) CNF requires $\neg$ to appear only in literals, so we "move $\neg$ inwards" by repeated application of the following standard equivalences: -

$\neg(\neg \alpha) \equiv \alpha$ (double - negation elimination)

$\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ (De Morgan)

$\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ (De Morgan)

In the example, we require just one application of the last rule.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \wedge P_{2,1}) \vee B_{1,1})$$

- 4) Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law, distributing $\vee$ over $\wedge$ wherever possible.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \wedge (\neg P_{2,1} \vee B_{1,1}) \vee B_{1,1})$$

Above sentence is now in CNF.

Resolution rule forms the basis for a family of complete inference procedures.

Refutation completeness:

It means that resolution can always be used to either confirm or refute a sentence but it cannot be used to enumerate true sentences.

Resolution algorithm

- 1) It takes input as knowledge base [a sentence in propositional logic] and α (it is query - which is a sentence in propositional logic).

BRUNN

A Guide for Engineering Students

- 2) Its output is true or false means that knowledge base do resolves α . It shows that knowledge base $\models \alpha$, for this it will be shown that $(KB \wedge \neg \alpha)$.
- 3) Steps are as follows,
- $(KB \wedge \neg \alpha)$ is first converted in to CNF.
 - Resolution rule is applied to the resulting clauses.
 - Each pair that contains complementary literals is resolved to produce new clause.
 - This new clause is added to set if it is not already present.
 - Step (ii) to (iv) are repeated until one of the two following situations occurs.
- a) There are no new clauses that can be added, in which case α does not entail β .

OR

- b) An application of the resolution rule derives the empty clause, in which case α entails β .

Completeness of resolution

The algorithm of resolution is complete. It means that it will surely produces a result checking all clauses in KB and all the clauses derivable from them. It finds closure set of clauses set which has all derivable clauses from existing clauses set. It checks the input query against the closure set.

Ground resolution theorem

It states that – "If a set of clauses is unsatisfiable, then the resolution closure of those clauses contains the empty clause".

The ground resolution theorem is called as completeness theorem for resolution.

Q.9 Explain forward chaining concept and algorithm along with Horn clause.

Ans. : The Horn Clause

- 1) Knowledge base contains simple restricted clauses called as Horn clauses. A Horn clause is a disjunction of literals of which at most one is positive.

- 2) Horn clause :
- It has atmost one positive literal.
 - Horn clause with exactly one positive literal are called definite literal.
 - The positive literal is called HEAD of the clause.
 - The negative literal is called as BODY of the clause.
 - A definite clause with no negative literals is called as fact.
 - Horn clause with no positive literals can be written as an implication whose conclusion is the literal false. Such horn clause can be used to represent integrity constraints (conditions) in the databases.
- 3) Inferencing with HORN clause - For inferencing with Horn clause forward and backward chaining methods are used.

Forward Chaining Method**The concept :**

- It is general concept of data-driven reasoning in which the focus of attention starts with known data.
- It can be used within an agent to derive conclusions from incoming percepts.
- It is used for determining whether a single proposition symbol Q (the query) is entailed by the knowledge base of Horn clauses.
- It initially starts from known facts (positive literals) in the knowledge base.
- If all the premises of an implication are known then its conclusion is added to the set of known facts.

For example :

If $L_{1,1}$ and breeze are known and $(L_{1,1} \wedge \text{Breeze}) \Rightarrow B_{1,1}$ is in the KB then $B_{1,1}$ can be added.

- 6) This process continues until the query Q is added or until no further inferences can be made.

Forward chaining algorithm :

Function PL-FC-ENTAILS ? (KB, q)

returns true or false

Inputs : KB, the knowledge base, a set of propositional Horn clauses, q, the query, a

proposition symbol.

Local variables : **Count**, a table, indexed by clause, initially the number of premises.
Inferred, a table, indexed by symbol, each entry initially false.
agenda, a list of symbols, initially the symbols known to be true in KB.

while agenda is not empty do

$p \leftarrow \text{POP}(\text{agenda})$

 unless inferred [p] do

 inferred [p] $\leftarrow$ true

 For each Horn clause (in whose premise p appears do decrement count [c])

 if count [c] = 0 then do

 if HEAD [c] = q then return true

 PUSH (HEAD [c], agenda)

 return false.

• Properties of forward chaining algorithm

1) Forward chaining algorithm is sound

- It means that every inference is essentially an application of modus ponens.

2) Forward chaining is complete.

- It means that every entailed atomic sentence will be derived. Algorithm constructs inferred table for final state. In this table each symbol inferred during process has true value and false for all other symbols.

3) For example :

Consider a simple knowledge base of Horn clause, as given below.

1. $A \Rightarrow Q$
2. $B \wedge C \Rightarrow A$
3. $D \wedge B \Rightarrow C$
4. $E \wedge A \Rightarrow B$
5. $E \wedge D \Rightarrow B$
6. E
7. D

AND - OR graph for above Horn clause knowledge base

- 1) Multiple links joined by arc indicate a conjunction.
- 2) Multiple links without an arc indicate a disjunction.
- 3) The links should be proved.
- 4) Inforce propagates up the graph.
- 5) Before proceeding ahead, from conjunction point, the propagation waits until all the conjuncts are known.

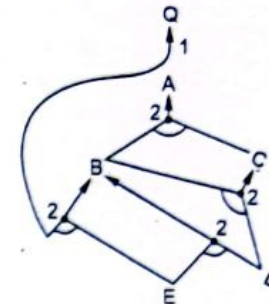


Fig. Q.9.1 AND-OR graph for Horn clause KB

• The order of clauses considered by forward chaining algorithm for inferencing query Q :

- 1) E and D are given true.
- 2) E and D implies B also B and A implies B.
- 3) D and B implies C.
- 4) B and C implies A.
- 5) A implies Q.
- 6) Therefore Q is true.

Q.10 Explain backward chaining concept and algorithm.

Ans. : The concept :

- 1) It works backwards from the query i.e if query Q is known to be true then no work is needed.
- 2) Algorithm finds those implications in the knowledge base that conclude Q.
If all the premises of one of those implications can be proved true (by backward chaining) then Q is true.
- 3) Backward chaining works down the graph starting at Q until it reaches a set of known facts that forms the basis for a proof.
- 4) Backward chaining is a form of goal-directed reasoning.
- 5) It is useful for answering questions such as "what action should I do now" ?

- 6) Cost of backward chaining is equal to or less than linear in the size of knowledge base, [As the process touches only relevant facts].

The example - [Referring to same knowledge base considered in forward chaining]

- 7) The order of clauses considered by backward chaining algorithm for inferecing query Q (in backward style) ---

1. Q is implied by A.
2. A is implied by B and C.
3. B is implied by E and A as well as E and D.
4. C is implied by E and D.
5. E and D are given true in knowledge base.
6. Therefore from Q we found implications sequence that conclude Q.

Therefore Q is true.

Q.11 Discuss backtracking algorithm used in propositional logic.

OR Discuss Davis Putnam Logmann-Loveland Algorithm (DPLL).

Ans. : The concept :

- 1) It takes input as a sentence in conjunctive normal form, which is a set of clauses.
- 2) It do recursive depth first enumeration of possible models.
- 3) It determines if input propositional logic sentence (in CNF) is satisfiable.

DPLL- A improved algorithm

This algorithm is improvement over earlier general inferencing algorithm for proposition logic (The TT - Entails algorithm). There are three improvements over truth table enumeration (TT - entails algorithm) as described below,

1) Early termination :

- i) A clause is true if any literal is true.
- ii) A sentence is false if any clause is false.

2) Pure symbol heuristic :

- i) **Pure symbol :** Pure symbol always appears with the same "sign" in all clauses.

Example :

In the three clauses $(A \vee \neg B)$, $(\neg B \vee \neg C)$, $(C \vee A)$, A and B are pure, C is impure.

- ii) Make a pure symbol literal true.

3) Unit clause heuristic :

- i) Unit clause : It is a clause which have only one literal in the clause.
- ii) The only literal in a unit clause must be true.

Example : If the model contains $B = \text{false}$, then $(B \vee \neg C)$ becomes a unit clause because it is equivalent to $(\text{false} \vee \neg C)$ or just $\neg C$.

The DPLL algorithm for checking satisfiability of a sentence in propositional logic

Function DPLL-SATISFIABLE (S) returns true or false

Inputs : S, a sentence in propositional logic.

Clause $\leftarrow$ The set of clauses in the CNF representation of S.

symbols $\leftarrow$ A list of the proposition symbols in S.

return DPLL (clauses, symbols, [])

DPLL algorithm (the searching procedure)

Function DPLL (clauses, symbols, model)

returns true or false

if every clause in clauses is

true in model then return true

if some clause in clauses is false

in model then return false

P, value $\leftarrow$ FIND-PURE-SYMBOL (symbols, clauses, model)

if P is non-null then return DPLL

(clauses, symbols, -P, EXTEND (P, value, model))

P value $\leftarrow$ FIND-UNIT-CLAUSE (clauses, model)

if P is non-null then return DPLL

(clauses, symbols-P, EXTEND (D, value, model))

P $\leftarrow$ FIRST (symbols); rest $\leftarrow$ REST (symbols)

return DPLL (clauses, rest,

EXTEND (P, true, model)) or DPLL (clauses, rest,

EXTEND (P, false, model)).

- Notes i) FIND-PURE-SYMBOL is a function which returns pure symbol (with value) or null.
- ii) FIND-UNIT-CLAUSE is a function which returns a unit clause (with value) or null.

Q.12 Explain local search algorithm for checking satisfiability . WALKSAT algorithm.

Ans. : The WALKSAT algorithm

[It is a "random walk" algorithm implementation for satisfiability].

Characteristics of WALKSAT algorithm :

- 1) It is local search algorithm.
- 2) Evaluation function :
The evaluation function is the min-conflict heuristic of minimizing the number of unsatisfied clauses.
- 3) It balances between greediness and randomness.
- 4) It is more useful when we expect a solution to exist. for e.g. 8 queen problem.
- 5) It is incomplete algorithm. It return satisfying model when it succeed but when it fails it has problem. When algorithm fails to satisfy the sentence it may enter into infinite loop because of infinite values of max-flips.
- 6) Local search algorithm like WALKSAT, cannot always detect unsatisfiability, which is very necessary for deciding entailment.

For example :

An agent cannot reliably use local search to prove that a square is safe in the Wumpus world.

Q.13 Explain circuit based agent in Wumpus world based on propositional logic.

Ans. : These are agents which evaluate logical expression directly in the form of circuits.

Characteristics of circuit based agent

- 1) Reflex agent with state.
- 2) Percepts are inputs to sequential circuit.
It has network of gates and registers.



- 3) Outputs are registers corresponding to actions.
- 4) Circuits are evaluated in a data flow model.
- 5) Value stored at each preposition symbol gives the truth value of the corresponding symbol at the current time t.
- 6) State estimation, is the general task of keeping track of environment state given a stream of percepts.
- 7) For logic based systems : Maintain a representation of the set of all logically possible world states, given axioms and percepts.
- 8) Basic trick : Successor-state axioms define truth of preposition at t+1 from prepositions at t.
- 9) For example :

$$\text{Alive}^t \Leftrightarrow \text{Scream}^t \wedge \text{Alive}^{t-1}$$

$$L_{1,1}^t \Leftrightarrow (L_{1,1}^{t-1} \wedge (\neg \text{Forward}^{t-1} \vee \text{Bump}^t)) \vee$$

$$(L_{1,2}^{t-1} \wedge (\text{Facing Down}^{t-1} \wedge \text{Forward}^{t-1}))$$

$$\vee (L_{2,1}^{t-1} \wedge (\text{Facing Left}^{t-1} \wedge \text{Forward}^{t-1}))$$

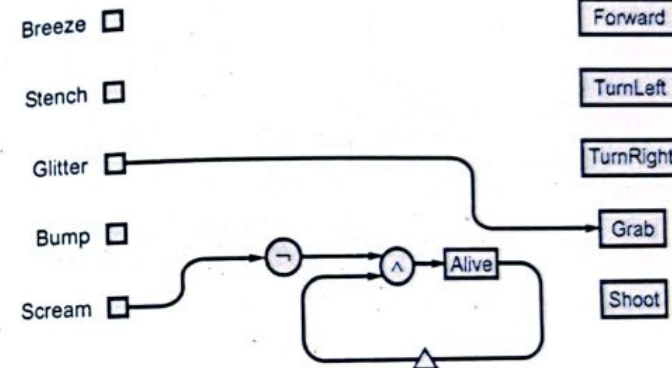


Fig. Q.13.1 Part of a circuit based agent for the wumpus world. Figure depicts circuit for grabbing the gold and the circuit for determining whether the wumpus is alive

Note

- Registers are shown as rectangles.
- One-step delays are shown as small rectangle.

Circuit based agent's location

- 1) One register for each L_{xy} .
- 2) Agent is in $[1, 1]$ at t if
 - The agent was in $[1, 1]$ at t^{-1} and has no moved, tried but bump.
 - It was at $[1, 2]$ facing down and moved forward.
 - It was at $[2, 1]$ facing left and move forward.

$$L_{1,1}^t < > L_{1,1}^{t-1} \wedge (\neg \text{Forward}^{t-1} \vee \text{Bump}^t) \\ \vee (L_{1,2}^{t-1} \wedge (\text{FacingDown}^{t-1} \wedge \text{Forward}^t)) \\ \vee (L_{2,1}^{t-1} \wedge (\text{FacingLeft}^{t-1} \wedge \text{Forward}^t))$$

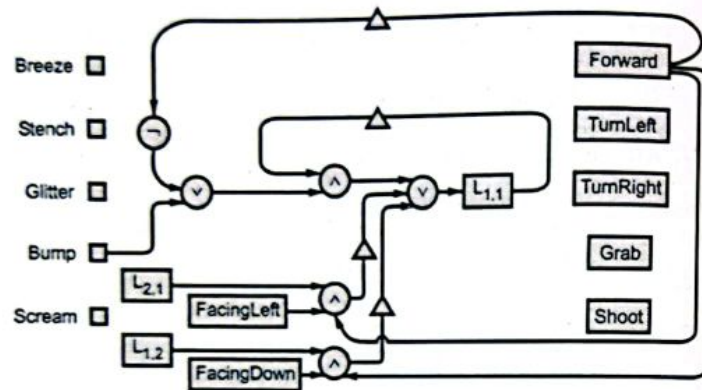


Fig. Q.13.2 The circuit for determining if the agent is at $[1, 1]$. Each location and orientation register has a similar circuit attached

A problem related to circuit base agent

What will be the initial truth value contained in $B_{4,4}$.

- The agent cannot set a truth value for it.
- Need to represent unknown, environment.
- Use of knowledge prepositions

$$K(B_{4,4}), K(\neg B_{4,4})$$

Q.14 Discuss comparative analysis of inference based agent and circuit based.

Ans. : 1) **Conciseness**

IBA - It require seperate copies of its knowledge base for every time step.

CBA - It do not require seperate copies of its knowledge base for every time step.

Both agents requires physics (expressed as sentences or circuits) for every square and therefore they are not suitable for large scale environment.

2) **Computational efficiency**

IBA - It takes exponential time for inferencing. (Time grows exponentially with respect to number of symbols)

CBA - It takes linear time for inferencing which is dependent on circuit size.

3) **Completeness**

IBA :

- 1) It is complete but with huge time requirements for completeness.
- 2) It also stores previous states/percept therefore memory requirement is also more.

CBA :

- 1) It is incomplete for various reasons :
 - a) Acyclicity restriction in circuit itself which is practically not possibles.
 - b) For a complete circuit CBA will take exponential time with respect to circuit size.
- 2) It forgets all previous states therefore can not draw conclusions based on previous states/percepts.

4) **Ease of construction**

IBA - As environment and conditions are simple and clear. It is easy to represent in propositional logic language.

CBA - The environment considered is limited therefore a circuit is small acyclic and semicomplete. Hence describing it is easy when relation between percepts and action is simple and straightforward. It is easy to describe and construct knowledgebase for such agents.

4.2 First-Order Logic, Representation Revisited, Syntax and Semantics of First-Order Logic, Using First-Order Logic, Knowledge Engineering in First-Order Logic

Q.15 What are higher-order logic ?

Ans. : Higher-Order Logic

- FOPL is called first-order because it allows quantifiers to range over objects (terms) but not properties, relations or functions applied to those objects.
- Second-order logic allows quantifiers to range over predicates and functions as well :

$$"X" Y [(x=y) \cup ("p p(x) \cup p(y))]$$

Says that two objects are equal if and only if they have exactly the same properties.

$$"f" g [(f=g) \cup ("X f(x) = g(x))]$$

Says that two functions are equal if and only if they have the same value for all possible arguments.
- Third-order would allow quantifying over predicates of predicates, etc.
 For example : A second-order predicate would be symmetric (p) stating that a binary predicate p represents a symmetric relation.
- Higher order logic views the relations and functions referred by first-order logic as objects in themselves.
- Higher order logic makes further ontological commitments.

Q.16 What is universal quantifier ?

Ans. : Universal Quantifier ($\forall$)

- i) It is represented using symbol ($\forall$)
 $\forall$ is pronounced as "for all".
- ii) The sentence formed using universal quantifier use a variable.
- **Variable :**
 1. It is term itself.
 2. It is represented using lowercase alphabets like x, y, z.

3. It can take up value from allowable objects, relation or function set.
4. A term without variable is called as ground term.
- iii) The sentence

$$\forall x P$$

Say that P is true for every object x, where P is a logical expression.

For example :

- a) "All Princess are Person" can be represented as,

$$\forall x \text{ Princess}(x) \Rightarrow \text{Person}(x)$$
- b) "If the universe of discourse is people, then this means that everyone is happy" can be represented as,

$$\forall x \text{ Happy}(x)$$

iv) ' $\forall$ ' can make statement about every object.

v) ' $\Rightarrow$ ' is natural connective to use with ' $\forall$ '.

vi) **Common mistake to avoid :** Do not use $\wedge$ as the main connective with $\forall$.

For example :

$$\forall x \text{ At}(x, \text{IIT}) \wedge \text{Smart}(x)$$

It means that "Everyone is at IIT and everyone is smart."

It is mistake because you wanted to say "everyone at IIT is smart."

Q.17 What is existential quantifier ?

Ans. : Existential Quantifier ($\exists$)

- i) These are used to make statement about some objects and not just for all.
 - ii) The objects are not named.
 - iii) The sentence, $\exists x P$ says that P is true for atleast one object x.
- For example :

a) "Princess Cindrella has crown on her head" can be expressed as,

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{Cindrella})$$

[Crown is a crown and it is on Cindrella's head]

b) "If the universe of discourse is people, then this means there is at least one happy person", can be expressed as,

$$\exists x \text{ Happy}(x)$$

iv) ' $\exists x$ ' it is pronounced as "There exists an x such that...."

OR

"For some x".

v) ' $\wedge$ ' is natural connective to be used with ' $\exists$ '.

vi) Common mistake to avoid : Do not use $\Rightarrow$ as the main connective with $\exists$.

For example :

$\exists x \text{ At } (x, \text{IIT}) \Rightarrow \text{Smart } (x)$ is true if there is anyone who is not at IIT.

Q.18 Translate these sentences into formulas in predicate logic.

1. John likes all kinds of food.
2. Apples are food.
3. Chicken is food.
4. Anything anyone eats and isn't killed-by is food.
5. Bill eats peanuts and is still alive.
6. Sue eats everything Bill eats.

Ans. : Translate sentences into formulas in predicate logic

1) John likes all kinds of food.

$\forall x \text{ Likes } (\text{John}, x) \wedge \text{Food } (x)$

OR / $\forall x \text{ food } (x) \rightarrow \text{like } (\text{John}, x)$

2) Apple are food.

$\text{Food } (\text{Apple})$

3) Chicken is food.

$\text{food } (\text{chicken})$

4) Anything anyone eats and isn't killed by is food.

$\forall x : \forall y : \text{eat } (x, y) \wedge \sim \rightarrow \text{food } (y)$

5) Bill eats Peanuts and is still alive.

$\text{eat } (\text{Bill}, \text{Peanut}) \wedge \text{alive } (\text{Bill})$

6) Sue eats everything Bill eats

$\forall x \text{ eat } (\text{Sue}, x) \rightarrow \text{eat } (\text{Bill}, x)$

Q.19 Convert the formulas derived in above question 18 into clauses. Prove that John likes peanuts using resolution.

Ans. : Conversion of formulas derived in Q.19.1 into clauses.

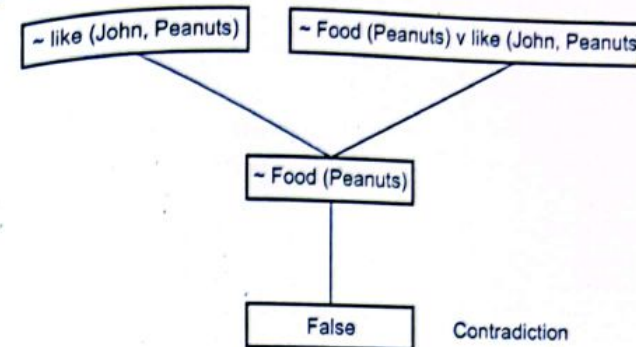


Fig. Q.19.1

1) $\forall x : \text{food } (x) \rightarrow \text{like } (\text{John}, x)$
 $\sim \text{food } (x) \vee \text{like } (\text{John}, x)$... (i)

2) In above equation (i) we substitute
 $x = \text{Peanuts}$

$\therefore \sim \text{food } (\text{Peanuts}) \vee \text{like } (\text{John}, \text{Peanuts})$... (ii)

We need to prove,

'John likes Peanuts' i.e. $\text{like } (\text{John}, \text{Peanuts})$

$\therefore$ We prove by resolution, we add negation of goal / result in our set of statements, $\sim \text{like } (\text{John}, \text{Peanuts})$... (iii)

Resolving equations (iii) and (ii)

Hence, goal $\sim \text{like } (\text{John}, \text{Peanuts})$ is false.

Hence, the proof $\text{like } (\text{John}, \text{Peanuts})$.

Q.20 Using the formulas derived in Q.18 prove John likes peanuts using forward chaining as well as backward chaining.

Ans. : Using wff derived in Q.18.

We prove now that,

$\Rightarrow$ 'John like Peanuts' using forward chaining

$\Rightarrow$ 'John like Peanuts' using backward chaining

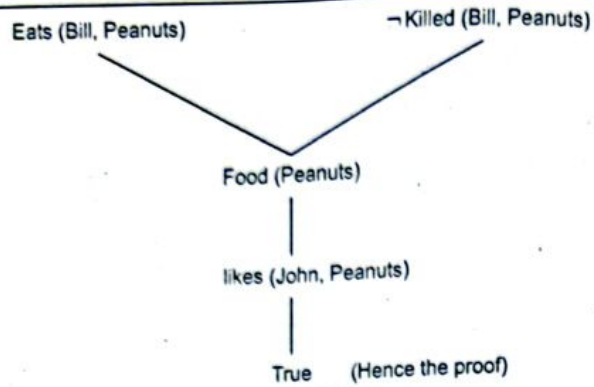


Fig. Q.20.1

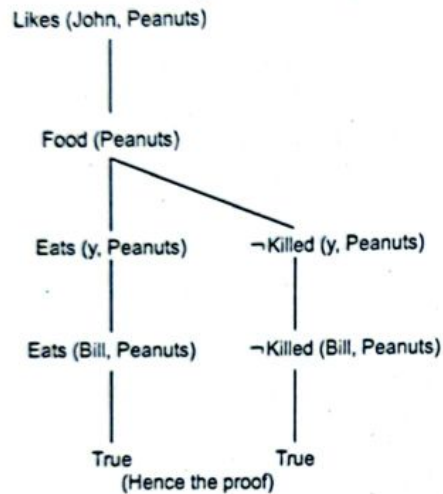


Fig. Q.20.2

Q.21 What is wrong with the following arguments ?

- Men are widely distributed over the earth
- Socrates is a man
- Therefore, Socrates is widely distributed over the earth.

How should the facts represented by these sentences be represented in logic so that this problem does not arise ?

Ans. : Consider argument,

⇒ Socrates is a man this can be represented as,
MAN (Socrates)

⇒ Men are widely distributed over the earth is represented as,

DISTRIBUTED - ON - EARTH - MAN

⇒ Therefore, Socrates is widely distributed over the earth is represented as,

DISTRIBUTED - ON - EARTH (Socrates)

In above case it fails to capture the relationship between any individual being 'man' and that individual being distributed on earth. To do that there is need of variable and quantification unless there is need to write separate statements about the 'distributionship' of 'every man' known present on earth.

⇒ The facts described by these sentences can be represented as a set of wff's in predicate logic as follows;

- Men are widely distributed over the earth
 $\forall x : \text{man}(x) \rightarrow \text{distributed-over-earth}(x)$
- Socrates is a man
MAN (Socrates)
- Socrates is widely distributed over the earth.
MAN (Socrates) $\rightarrow$ distributed-over-earth (Socrates).

Q.22 Explain the algorithm of predicate logic resolution.

Ans. : The resolution inference rule :

- Two clauses, which are assumed are to be standardized apart so that they share no variables, can be resolved if they contain complementary literals.
- First order literals are complementary if one unifies with the negation of the other.
- The resolution inference rule is

$$\frac{l_1, V \dots V l_k, \quad m_1 V \dots V m_n}{\text{SUBST}(\theta, l_1, V \dots V l_{i-1} V, l_{i+1} V \dots V l_k V_m, V \dots V m_{j-1} V m_{j+1} V \dots V m_n)}$$